

# Chain Poker Genesis

## Official Updated Document

### Engine Governance Layer (EGL) — Engine of Engines

---

## 1. General Definition

The Engine of Engines is the technical governance layer of Chain Poker Genesis responsible for registering, classifying, auditing, evaluating, versioning, and enabling compatible functional engines within the protocol ecosystem.

Its purpose is not to execute games, manage funds, create wallets, settle payments, or interfere with the operational consensus of a table.

The Engine of Engines exists to enable ecosystem evolution without modifying the protocol core and without depending on a centralized maintenance authority.

---

## 2. Nature of the Engine of Engines

The Engine of Engines is an independent component from all other system engines.

It is not part of:

- Group Table Wallet Creation Engine.
- Monetary Settlement Engine.
- Poker Rules Engine.
- Card Distribution Engine.
- Table Position Engine.
- Viewer Engine.
- Lightning Engine.
- Remote Virtual Node Engine.

Its only responsibility is to manage the catalog of available capabilities within the ecosystem.

---

## 3. Fundamental Principles

### 3.1 Absolute Separation

The Engine of Engines has no access to:

- private keys;
  - Taproot signatures;
  - Lightning channels;
  - table balances;
  - player funds;
  - internal states of an active poker hand.
- 

### 3.2 No Operational Authority

The Engine of Engines cannot:

- force hot migrations;
  - modify active tables;
  - alter already signed consensus;
  - change session rules;
  - replace an engine without explicit protocol and table authorization.
- 

### 3.3 Verifiable Evolution

The ecosystem evolves through the incorporation of new verified engines, not through arbitrary replacement of the base protocol.

---

## 4. Functions of the Engine of Engines

The Engine of Engines performs the following functions:

- register compatible engines;
- receive proposals for new engines;
- distribute technical information to nodes;
- coordinate review and audit periods;
- maintain version history;
- manage public reputation;

- classify engine maturity states;
  - enable consensual adoption by new tables.
- 

## 5. Official Engine Registry

The Engine of Engines maintains a registry called:

ENGINE\_REGISTRY

Each registry entry represents an engine recognized by the protocol.

### 5.1 Minimum Registry Fields

Each engine must include:

- `engine_id`
- `engine_type`
- `name`
- `version`
- `code_hash`
- `status`
- `interfaces`
- `audit_history`
- `reputation_metrics`
- `compatibility_notes`

### 5.2 Conceptual Example

```
{
  "engine_id": "CPG-ENG-GROUP-WALLET-TAPROOT",
  "engine_type": "GROUP_TABLE_WALLET_ENGINE",
  "name": "Taproot Group Table Wallet Engine",
  "version": "1.0.0",
  "code_hash": "sha256:...",
  "status": "AVAILABLE",
  "interfaces": [
    "CPG_ENGINE_INTERFACE_V1"
  ],
  "audit_history": [],
  "reputation_metrics": {
    "security_score": 0,
    "compatibility_score": 0,
```

```
"review_count": 0,  
"reputation_score": 0  
}  
}
```

---

## 6. Engine Lifecycle

Every engine recognized by the protocol follows a formal lifecycle.

### 6.1 Allowed States

PROPOSED

↓

TECHNICAL\_REVIEW

↓

SECURITY\_AUDIT

↓

PUBLIC\_EVALUATION

↓

IMPLEMENTATION\_CANDIDATE

↓

APPROVED

↓

AVAILABLE

↓

DEPRECATED

Additional states:

REJECTED

SUSPENDED

---

## 7. Meaning of Each State

### **PROPOSED**

The engine has been submitted to the ecosystem but has not yet been validated.

---

### **TECHNICAL\_REVIEW**

The architecture, interfaces, documentation, and compatibility are reviewed.

---

### **SECURITY\_AUDIT**

Cryptographic security, state management, and code integrity are evaluated.

---

### **PUBLIC\_EVALUATION**

The ecosystem may review, comment, and rate the engine.

---

### **IMPLEMENTATION\_CANDIDATE**

The engine is suitable for testing environments or experimental adoption.

---

### **APPROVED**

The engine has passed technical validation and fulfilled protocol requirements.

---

## AVAILABLE

The engine is officially available for selection by compatible new tables.

---

## DEPRECATED

The engine remains historically recognized but is no longer recommended for new tables.

---

## REJECTED

The proposal was discarded due to incompatibility, security concerns, or failure to meet requirements.

---

## SUSPENDED

The engine is temporarily disabled due to security, integrity, or review issues.

---

## 8. New Engine Proposal

Any developer or team may submit an engine through:

ENGINE\_PROPOSAL

### 8.1 Required Proposal Information

The proposal must include:

- engine name;
- engine type;
- version;
- functional description;
- architecture;
- supported interfaces;
- dependencies;
- technical documentation;

- verifiable code hash;
  - security model;
  - performed tests;
  - Chain Poker Genesis compatibility criteria.
- 

## 9. Review Notification

When a proposal enters the evaluation process, the Engine of Engines issues:

ENGINE\_REVIEW\_NOTIFICATION

Distributed information includes:

- engine identifier;
  - proposed version;
  - engine type;
  - code hash;
  - documentation;
  - review start date;
  - evaluation period;
  - current review status.
- 

## 10. Technical Audit

Every engine under review may undergo technical auditing.

### 10.1 Evaluated Areas

- security;
- compatibility;
- code quality;
- stability;
- performance;
- maintainability;
- consistency with protocol interfaces.

### 10.2 Audit Purpose

The audit does not replace table consensus or functional validation.

Its purpose is to determine whether the engine can advance through the ecosystem lifecycle.

---

## 11. Public Reputation System

The Engine of Engines maintains public metrics reflecting accumulated trust in an engine.

### 11.1 Metrics

Examples:

- `security_score`
- `compatibility_score`
- `review_count`
- `reputation_score`

### 11.2 Reputation Function

Reputation:

- does not replace technical audits;
  - does not independently authorize adoption;
  - provides additional ecosystem information;
  - helps tables select more reliable components.
- 

## 12. Implementation Period

When an engine passes technical review and required audits, it may enter:

IMPLEMENTATION\_CANDIDATE

During this period:

- it may be tested;
  - it may coexist with previous engines;
  - it may be used by experimental tables;
  - it does not automatically replace previous versions.
-

## 13. Engine Activation

An engine can become:

AVAILABLE

when it satisfies:

- technical compatibility;
- successful review;
- security validation;
- sufficient reputation;
- approval according to protocol-defined mechanisms.

Once available, it is listed in the official registry and may be selected by new tables that explicitly accept it.

---

## 14. Versioning and Historical Immutability

Each engine maintains its own historical identity through:

- version;
- hash;
- associated audits;
- current status;
- change history.

A new version does not modify previous versions.

Example:

ENGINE\_v1  
ENGINE\_v2  
ENGINE\_v3

Each version remains independently identifiable within the protocol history.

---

## 15. Table Engine Selection

The Engine of Engines does not select engines.

Selection belongs to the table consensus process.

During table creation, consensus may define:

- wallet engine;
- settlement engine;
- poker rules engine;
- card engine;
- viewer engine;
- any other compatible approved engine.

The Engine of Engines only exposes what components exist, their status, and their adoption conditions.

---

## 16. Relationship with the Group Table Wallet Creation Engine

The Engine of Engines is independent from the Group Table Wallet Creation Engine.

Therefore:

The Engine of Engines:

- does not create wallets;
- does not manage Taproot;
- does not sign funds;
- does not determine Lightning routes;
- does not perform settlement.

The Group Table Wallet Creation Engine may be registered and evaluated by the Engine of Engines, but both components remain separate.

---

## 17. Protocol Evolution

Chain Poker Genesis evolves through the incorporation of verifiable capabilities.

Model:

BASE PROTOCOL

+

AUDITED ENGINES

=

EVOLVING ECOSYSTEM

This allows different engine generations to coexist without breaking compatibility between new and existing tables.

---

## 18. Formal Limitations

The Engine of Engines cannot:

- modify active tables;
  - change existing consensus;
  - manage funds;
  - issue cryptographic signatures;
  - create wallets;
  - select engines unilaterally;
  - replace engines without validation;
  - interfere with gameplay decisions;
  - modify poker rules;
  - alter hand execution sequences.
- 

## 19. Coexistence Principle

The protocol allows multiple engine generations to coexist simultaneously.

Example:

One table may use a stable engine version while another table uses an experimental engine, provided both are registered and selected through valid consensus.

This prevents forced forks and allows gradual ecosystem evolution.

---

## 20. No-Hard-Fork Evolution Principle

The Engine of Engines exists to reduce the need for protocol hard forks.

Technological evolution occurs through:

- review;
- auditing;
- reputation;
- voluntary adoption;
- declared compatibility.

Not through centralized control or mandatory migration.

---

## 21. Engine of Engines State Machine

Formal lifecycle:

PROPOSED

↓

TECHNICAL\_REVIEW

↓

SECURITY\_AUDIT

↓

PUBLIC\_EVALUATION

↓

IMPLEMENTATION\_CANDIDATE

↓

APPROVED

↓

AVAILABLE

Exit states:

REJECTED

SUSPENDED

DEPRECATED

---

## 22. Operational Summary

The Engine of Engines allows Chain Poker Genesis to:

- grow without breaking its core;
- incorporate new technologies through verification;
- preserve technical history;
- allow competition between engines;
- maintain independence between tables;
- guarantee conscious and auditable component selection.

Its role is structural, not operational.

It is the layer that transforms Chain Poker Genesis into an evolving ecosystem rather than closed software requiring forced updates.

## Chain Poker Genesis

## Documento Oficial Actualizado

## Motor de Motores (Engine Governance Layer - EGL)

---

# 1. Definición General

El Motor de Motores es la capa de gobernanza técnica de Chain Poker Genesis encargada de registrar, clasificar, auditar, evaluar, versionar y habilitar motores funcionales compatibles con el protocolo.

Su función no es ejecutar partidas, administrar fondos, crear billeteras, liquidar apuestas ni intervenir en el consenso operativo de una mesa.

El Motor de Motores existe para permitir la evolución del ecosistema sin alterar el núcleo del protocolo ni depender de una autoridad central de mantenimiento.

---

## 2. Naturaleza del Motor de Motores

El Motor de Motores es un componente independiente del resto de motores del sistema.

No forma parte de:

- Motor de Creación de Billetera de Mesa Grupal.
- Motor de Liquidación Monetaria.
- Motor de Reglas de Poker.
- Motor de Repartición de Cartas.
- Motor de Posiciones de Mesa.
- Motor de Visor.
- Motor Lightning.
- Motor de Nodos Virtuales Remotos.

Su única responsabilidad es administrar el catálogo de capacidades disponibles para el ecosistema.

---

## 3. Principios Fundamentales

### 3.1 Separación absoluta

El Motor de Motores no tiene acceso a:

- claves privadas;
- firmas Taproot;
- canales Lightning;
- balances de mesa;

- fondos de jugadores;
- secuencias internas de una mano activa.

## 3.2 No intervención operacional

El Motor de Motores no puede:

- forzar migraciones en caliente;
- modificar mesas activas;
- cambiar consensos ya firmados;
- alterar reglas de una sesión en curso;
- sustituir un motor sin decisión expresa del protocolo y de la mesa correspondiente.

## 3.3 Evolución verificable

La evolución del ecosistema ocurre por incorporación de motores nuevos, no por sustitución arbitraria del protocolo base.

---

# 4. Funciones del Motor de Motores

El Motor de Motores tiene como funciones principales:

- registrar motores compatibles;
  - recibir propuestas de nuevos motores;
  - distribuir información técnica a los nodos;
  - coordinar periodos de revisión y auditoría;
  - mantener historial de versiones;
  - administrar reputación pública;
  - clasificar estados de madurez de cada motor;
  - permitir adopción consensuada por mesas nuevas.
- 

# 5. Registro Oficial de Motores

El Motor de Motores mantiene un registro denominado:

ENGINE\_REGISTRY

Cada entrada del registro representa un motor reconocido por el protocolo.

## 5.1 Campos mínimos del registro

Cada motor debe incluir, como mínimo:

- `engine_id`
- `engine_type`
- `name`
- `version`
- `code_hash`
- `status`
- `interfaces`
- `audit_history`
- `reputation_metrics`
- `compatibility_notes`

## 5.2 Ejemplo conceptual

```
{
  "engine_id": "CPG-ENG-GROUP-WALLET-TAPROOT",
  "engine_type": "GROUP_TABLE_WALLET_ENGINE",
  "name": "Taproot Group Table Wallet Engine",
  "version": "1.0.0",
  "code_hash": "sha256:...",
  "status": "AVAILABLE",
  "interfaces": [
    "CPG_ENGINE_INTERFACE_V1"
  ],
  "audit_history": [],
  "reputation_metrics": {
    "security_score": 0,
    "compatibility_score": 0,
    "review_count": 0,
    "reputation_score": 0
  }
}
```

---

# 6. Ciclo de Vida de un Motor

Todo motor reconocido por el protocolo atraviesa un ciclo de vida formal.

## 6.1 Estados permitidos

PROPOSED  
TECHNICAL\_REVIEW  
SECURITY\_AUDIT  
PUBLIC\_EVALUATION  
IMPLEMENTATION\_CANDIDATE  
APPROVED  
AVAILABLE  
DEPRECATED  
REJECTED  
SUSPENDED

## **6.2 Significado de los estados**

### **PROPOSED**

El motor ha sido presentado al ecosistema, pero aún no está validado.

### **TECHNICAL\_REVIEW**

Se revisan la arquitectura, interfaces, documentación y compatibilidad.

### **SECURITY\_AUDIT**

Se verifica la seguridad criptográfica, el manejo de estados y la integridad del código.

### **PUBLIC\_EVALUATION**

La comunidad puede revisar, comentar y calificar el motor.

### **IMPLEMENTATION\_CANDIDATE**

El motor es apto para entornos de prueba o adopción experimental.

### **APPROVED**

El motor fue validado técnicamente y aprobado bajo los criterios del protocolo.

### **AVAILABLE**

El motor está disponible para ser seleccionado por nuevas mesas compatibles.

### **DEPRECATED**

El motor sigue siendo reconocible por el sistema, pero ya no se recomienda para nuevas mesas.

## **REJECTED**

La propuesta fue descartada por incompatibilidad, riesgo o incumplimiento técnico.

## **SUSPENDED**

El motor queda temporalmente deshabilitado por razones de seguridad, integridad o revisión pendiente.

---

# **7. Propuesta de Nuevo Motor**

Cualquier desarrollador o equipo puede presentar un motor mediante un objeto de propuesta formal:

ENGINE\_PROPOSAL

## **7.1 Contenido mínimo de la propuesta**

La propuesta debe incluir:

- nombre del motor;
- tipo de motor;
- versión;
- descripción funcional;
- arquitectura;
- interfaces soportadas;
- dependencias;
- documentación técnica;
- hash verificable del código;
- modelo de seguridad;
- pruebas realizadas;
- criterio de compatibilidad con Chain Poker Genesis.

## **7.2 Objetivo de la propuesta**

La propuesta no busca aprobar de inmediato un motor, sino iniciarlo dentro del proceso de evaluación del ecosistema.

---

# **8. Notificación de Revisión**

Cuando una propuesta es aceptada para análisis inicial, el Motor de Motores emite una notificación pública o distribuida a los nodos.

Objeto asociado:

ENGINE\_REVIEW\_NOTIFICATION

## 8.1 Información transmitida

- identificador del motor;
  - versión propuesta;
  - tipo de motor;
  - hash del código;
  - documentación asociada;
  - fecha de inicio de revisión;
  - duración del periodo de evaluación;
  - estado de revisión.
- 

# 9. Auditoría Técnica

Todo motor en revisión puede ser sometido a auditoría técnica.

## 9.1 Aspectos evaluados

- seguridad;
- compatibilidad;
- calidad del código;
- estabilidad;
- rendimiento;
- mantenibilidad;
- consistencia con las interfaces del protocolo.

## 9.2 Finalidad de la auditoría

La auditoría no sustituye el consenso de una mesa ni la validación funcional del motor en producción.

Su objetivo es determinar si el motor puede avanzar a estados más maduros dentro del ecosistema.

---

## 10. Sistema de Reputación Pública

El Motor de Motores mantiene métricas públicas que reflejan la confianza acumulada en un motor.

### 10.1 Métricas típicas

- `security_score`
- `compatibility_score`
- `review_count`
- `reputation_score`

### 10.2 Función de la reputación

La reputación:

- no reemplaza auditorías;
  - no autoriza por sí sola la adopción;
  - sí influye en la percepción técnica y comunitaria del motor;
  - ayuda a las mesas a seleccionar componentes más confiables.
- 

## 11. Periodo de Implementación

Cuando un motor supera la revisión técnica y la auditoría requerida, puede pasar a estado:

`IMPLEMENTATION_CANDIDATE`

En este punto:

- puede ser probado;
  - puede coexistir con motores anteriores;
  - puede ser usado en mesas experimentales;
  - no reemplaza automáticamente versiones previas.
- 

## 12. Activación de un Motor

Un motor puede pasar a estado:

AVAILABLE

cuando cumple simultáneamente con:

- compatibilidad técnica;
- revisión satisfactoria;
- auditoría de seguridad;
- reputación suficiente;
- aprobación bajo los mecanismos del protocolo.

Una vez disponible, el motor queda listado en el registro oficial y puede ser seleccionado por mesas nuevas que lo acepten explícitamente.

---

## 13. Versionado e Inmutabilidad Histórica

Cada motor conserva su identidad histórica mediante:

- versión;
- hash;
- auditoría asociada;
- estado actual;
- historial de cambios.

Una nueva versión no modifica a las versiones anteriores.

Ejemplo conceptual:

ENGINE\_v1  
ENGINE\_v2  
ENGINE\_v3

Cada versión conserva su propia trazabilidad dentro del ecosistema.

---

## 14. Selección por parte de las Mesas

El Motor de Motores no elige motores por su cuenta.

La selección pertenece al proceso de consenso de mesa.

Cuando una mesa se crea, el consenso puede fijar:

- motor de billetera;
- motor de liquidación;
- motor de reglas;
- motor de mazo;
- motor de visor;
- cualquier otro motor compatible y aprobado.

El Motor de Motores únicamente expone qué componentes existen, en qué estado están y bajo qué condiciones pueden ser adoptados.

---

## 15. Relación con el Motor de Creación de Billetera de Mesa Grupal

El Motor de Motores es independiente del motor de creación de billetera.

Por lo tanto:

- no crea billeteras;
- no administra Taproot;
- no firma fondos;
- no decide rutas Lightning;
- no liquida pagos;
- no sustituye al motor de billetera.

El motor de billetera puede ser uno de los motores registrados y evaluados por el Motor de Motores, pero ambos componentes permanecen separados.

---

## 16. Relación con la Evolución del Protocolo

Chain Poker Genesis evoluciona mediante incorporación de capacidades verificables.

Modelo general:

PROTOCOLO BASE

+

MOTORES AUDITADOS

=

ECOSISTEMA EVOLUTIVO

Esto permite que existan motores distintos para distintos momentos históricos del protocolo sin romper compatibilidad entre mesas nuevas y mesas ya existentes.

---

## 17. Limitaciones Formales

El Motor de Motores no puede:

- modificar mesas activas;
  - cambiar consensos existentes;
  - administrar fondos;
  - emitir firmas criptográficas;
  - crear billeteras;
  - seleccionar motores unilateralmente;
  - reemplazar motores sin proceso de validación;
  - intervenir en decisiones de juego;
  - alterar reglas de poker;
  - alterar la secuencia de una mano.
- 

## 18. Principio de Coexistencia

El protocolo permite que distintas generaciones de motores coexistan simultáneamente.

Ejemplo:

Una mesa puede usar un motor estable y otra mesa un motor experimental, siempre que ambos estén debidamente registrados y seleccionados por consenso.

Esto evita bifurcaciones forzadas y permite una evolución gradual del ecosistema.

---

## 19. Principio de No Bifurcación

El Motor de Motores existe para reducir la necesidad de bifurcaciones duras.

La evolución tecnológica del protocolo se realiza por:

- revisión;
- auditoría;
- reputación;

- adopción voluntaria;
- compatibilidad declarada.

No por imposición central ni por migración obligatoria de todo el sistema.

---

## 20. Máquina de Estados del Motor de Motores

El ciclo formal puede representarse así:

```
PROPOSED
↓
TECHNICAL_REVIEW
↓
SECURITY_AUDIT
↓
PUBLIC_EVALUATION
↓
IMPLEMENTATION_CANDIDATE
↓
APPROVED
↓
AVAILABLE
```

Estados terminales o de salida:

```
REJECTED
SUSPENDED
DEPRECATED
```

---

## 21. Resumen Operativo

El Motor de Motores es la capa que permite a Chain Poker Genesis:

- crecer sin romper su núcleo;
- incorporar nuevas tecnologías de forma verificable;
- mantener historial técnico;
- permitir competencia entre motores;
- preservar independencia entre mesas;

- garantizar selección consciente y auditable de componentes.

Su papel no es operativo sino estructural.

Es la capa que convierte al protocolo en un ecosistema evolutivo y no en un software cerrado.